Deepspace Defenders — Space Shooter Game

A Semester Project Report

Course:	CSE142 Object Oriented Programming Techniques
Semester:	Spring 2026
Instructor:	Behraj Khan
Due Date:	10th May 2026
Author(s):	Hamza Faisal — ERP: 32406
	Hunain Adnan — ERP: 32397
	Burhanuddin Murtaza Patanwala — ERP: 32400

Department of Computer Science
Institute of Business Administration, Karachi

May 10, 2026

Abstract

Deepspace Defenders is a fully playable 2D space shooter game developed in C++ using the SFML (Simple and Fast Multimedia Library) graphics framework. The player controls a spaceship and must survive waves of progressively harder alien enemies across three hand-crafted levels, each culminating in a boss fight.

We built this game to apply the OOP concepts from our course in a real project. Instead of just writing small examples, we wanted something that actually runs and uses all the concepts together. The architecture uses three inheritance hierarchies — one each for bullet types, enemy types, and game levels — all rooted at abstract base classes. A template class called `ObjectPool` manages all live game objects without any dynamic allocation during gameplay, which avoids memory leaks. The player class uses encapsulation to protect its internal state. Score is added through an overloaded operator, and custom exception classes handle asset loading failures and invalid level transitions.

The result is a smooth, 60-fps game that keeps data, logic, and rendering in separate, independent components — showing in practice that OOP makes code cleaner, safer, and easier to extend.

Keywords: Object-Oriented Programming, C++, SFML, Inheritance, Polymorphism, Encapsulation, Templates, Operator Overloading, Exception Handling, Game Development.

Contents

1 Introduction

1.1 Background

We chose to make a space shooter for our OOP project. A game is a good fit because everything in it — the player, enemies, bullets, levels, particles — can be modelled as a class, which is exactly what OOP is about. Each thing owns its own data and handles its own behaviour.

The space shooter genre works well for a semester project. It is simple enough to finish in time, but complex enough to actually need every major OOP concept: inheritance for different enemy types, polymorphism so the game loop does not need to know which level is loaded, encapsulation to protect the player's health, templates for a memory pool that works with multiple types, and operator overloading to add score with `+=`.

1.2 Problem Statement

We needed to build a game where:

- Multiple enemy *types* (small, medium, boss) share common logic but differ in stats and shooting patterns.
- Multiple *levels* each define their own wave schedule but share common rendering and loading logic.
- Multiple *bullet types* (player, enemy, boss beam) are distinguishable without scattering `if/else` chains everywhere.
- Hundreds of live objects (up to 700 bullets, 80 enemies, 1000 particles) are managed without allocating or freeing memory every frame — which would slow the game down and cause crashes.
- The game flows through distinct screens (Home, Level Select, Playing, Paused, Game Over, Level Complete) in a predictable, bug-free way.

1.3 Objectives

1. Design three independent inheritance hierarchies (bullets, enemies, levels) that are easily extendable.
2. Implement a generic `ObjectPool` template that recycles objects in a fixed array, avoiding runtime memory allocation.
3. Encapsulate all player state inside the `Player` class so no other code can accidentally corrupt it.
4. Use virtual functions and abstract classes so that the main game loop does not need to know the concrete type of each object.
5. Demonstrate operator overloading, static class members, and custom exceptions as part of the design — not as add-ons.
6. Deliver a complete, playable game: three levels, a boss fight, a HUD, a home screen, and a level-select screen.

1.4 Scope and Limitations

In scope: Three levels, three enemy types, four bullet types, a particle explosion system, a scrolling star background, background music (OGG format via SFML Audio), a home screen, a level-select screen, pause/resume, game over, and level-complete overlays.

Out of scope: Saved high scores (no file I/O at runtime), multiplayer, sound effects, and power-ups. These are possible future extensions.

2 OOP Design

2.1 Project File Structure

The project has 12 source files. Each file only depends on files that appear above it in the list. This order also defines the recommended coding sequence — write them top to bottom and every dependency will already exist when you need it.

Table 1: Source files in dependency order

Layer	File	Purpose
1	`Utils.h`	Pure math helpers (vector length, normalise, lerp, clamp, random)
1	`BulletType.h`	Abstract bullet category class + 4 concrete subclasses + `BulletTypeRegistry`
1	`EnemyType.h`	Abstract enemy profile class + 3 concrete subclasses + `EnemyTypeRegistry`
1	`GameState.h`	Game-screen state value class (constants defined in `main.cpp`)
2	`GameData.h`	Data structs (Bullet, Enemy, Particle, Star, SpawnEvent, SpawnJob) + ObjectPool template
3	`Player.h`	Player class (fully inline, no .cpp)
4	`Level.h`	Abstract Level base class + LevelOne, LevelTwo, LevelThree
4	`Level.cpp`	Level method implementations and wave definitions
5	`Game.h`	Game class declaration
5	`Game.cpp`	Core logic: init, update, collision detection, enemy spawning
5	`Game_graphics.cpp`	All rendering methods (render, renderBullets, renderEnemies, HUD, overlays)
6	`main.cpp`	Defines all static member variables; instantiates registries; game entry point

2.2 Class Hierarchy

There are three inheritance hierarchies and one composition hub (`Game`). Figures 1 and 2 show the relationships.

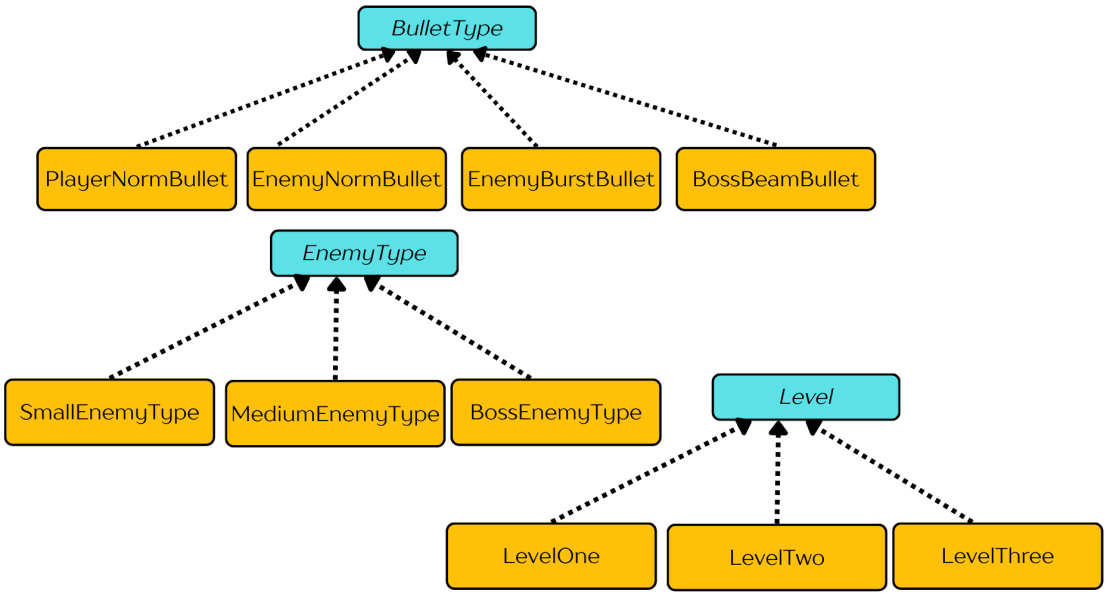

Figure 1: Three inheritance hierarchies (BulletType, EnemyType, Level). Arrows point from child to parent (*"inherits from"*). Shaded boxes are abstract bases; plain boxes are concrete subclasses.

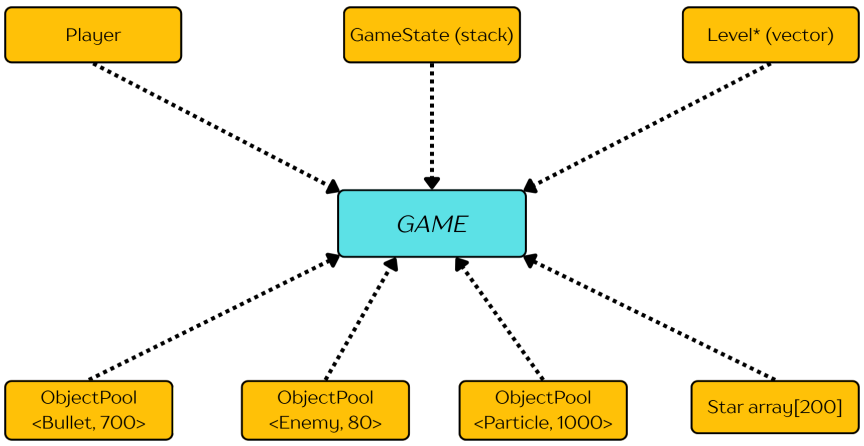

Figure 2: Composition: the `Game` class *owns* (has-a) all subsystems. Arrows labelled `has-a` mean "contains as a member variable".

2.3 Class and Struct Descriptions

2.3.1 Classes (with behaviour)

Table 2: Classes and their responsibilities

Class	Responsibility	Key members
`BulletType`	Identifies a bullet's category and owner	`id`, `playerBullet`, `getId()`, `isPlayerBullet()`, static singletons `PlayerNorm`, `EnemyNorm`, `EnemyBurst`, `BossBeam`
`PlayerNormBullet`	Player's straight-up shot	Inherits BulletType with id=0, playerBullet=true
`EnemyNormBullet`	Basic aimed enemy shot	id=3, playerBullet=false
`EnemyBurstBullet`	Spread-fire enemy shot	id=4, playerBullet=false
`BossBeamBullet`	Boss beam projectile	id=5, playerBullet=false
`BulletTypeRegistry`	Owns and initialises all four bullet-type singletons	Private members: one instance of each concrete subclass; constructor wires static pointers; destructor clears them to `nullptr`; copy disabled (`= delete`)
`EnemyType`	Stores the stat profile for one enemy category	`maxHp`, `shootInterval`, `scoreValue`, `moveSpeed`, `radius`, `kind`; static singletons `Small`, `Medium`, `Boss`
`SmallEnemyType`	Fast, weak enemy	30 HP, interval 2.2s, radius 16px, worth 50 pts
`MediumEnemyType`	Tanky spread-shooter	80 HP, interval 1.8s, radius 22px, worth 120 pts
`BossEnemyType`	End-of-level boss	1200 HP, interval 0.8s, radius 60px, worth 2000 pts
`EnemyTypeRegistry`	Owns and initialises all three enemy-type singletons	Same RAII pattern as `BulletTypeRegistry`; constructor assigns `Small`, `Medium`, `Boss` pointers; destructor sets them to `nullptr`; copy disabled
`GameState`	Tracks which screen is active	`id`; `operator==`, `operator!=`; static constants `Home`, `LevelSelect`, `Playing`, `Paused`, `GameOver`, `LevelComplete`
`Player`	Encapsulates all player state and behaviour	pos, hp, maxHp, speed, lives, score, timers; movement, clamping, damage, healing, score accumulation
`Level`	Abstract base for a playable level	name, desc, events (SpawnEvent list), planetTexture; pure virtual `buildWaves()`, `getDuration()`, `getPlanetTexturePath()`
`LevelOne/Two/Three`	Concrete level implementations	Override `buildWaves()` with a timed enemy schedule
`ObjectPool<T,N>`	Fixed-size object recycler	`items[N]`; `alloc()`, `clearAll()`, `countActive()`
`Game`	Central orchestrator	Owns all subsystems; implements init, run, update, collision, spawning; includes `sf::Music` for background audio; rendering split into `Game_graphics.cpp`

2.3.2 Data Structs (plain data containers)

Table 3: Data structs used as game objects

Struct	Represents	Key fields
`Bullet`	One active projectile	pos, vel, dmg, `BulletType*` type, on
`Enemy`	One active enemy	pos, vel, hp, maxHp, `EnemyType*` type, shootTimer, shootInterval, path, pathIndex, angle, pulse, scoreValue, moveSpeed, on
`Particle`	One explosion spark	pos, vel, colorStart, colorEnd, life, maxLife, size, on
`SpawnEvent`	A scheduled group spawn	time, etype, startPos, path, count, xSpacing, delay, fired
`SpawnJob`	One pending individual spawn	etype, startPos, delayRemaining, path, hp, maxHp, shootInterval, scoreValue, moveSpeed
`Star`	One background star	pos, speed, brightness, size

Why separate Struct from Class? In this project, a *struct* is just a passive data container — it holds values and has a default constructor, nothing else. A *class* has active behaviour: it enforces rules, provides getters and setters, and contains logic. Keeping data structs simple and putting behaviour in classes makes the code easier to follow.

2.4 OOP Concepts Summary

Table 4: OOP concepts used and where they appear

OOP Concept	**Where and how it is used**
Encapsulation	`Player` class: all fields are `private`; external code uses getters and setters only
Inheritance	Three hierarchies: BulletType, EnemyType, Level (each abstract base + concrete subclasses)
Polymorphism	`Level::buildWaves()`, `getDuration()`, and `getPlanetTexturePath()` are pure virtual; `Game` calls them without knowing the concrete type
Abstract Classes	`BulletType`, `EnemyType`, and `Level` cannot be instantiated directly because they have pure virtual destructors or methods
Templates	`ObjectPool<T,N>` works for `Bullet`, `Enemy`, and `Particle` using the same code
Operator Overloading	`Player::operator+=(int)` adds score; `GameState::operator==` and `operator!=` compare states
Static Members	`BulletType::PlayerNorm`, `EnemyType::Small`, `GameState::Home` etc. — global singletons accessed through the class name
Exception Handling	`AssetLoadException` and `InvalidLevelException` (custom classes inheriting `std::runtime_error`)
Virtual Destructor	All abstract base classes (`BulletType`, `EnemyType`, `Level`) declare `virtual ~Base()` so derived objects are safely deleted via a base pointer
Composition (has-a)	`Game` contains `Player`, three `ObjectPool` instances, a `vector<Level*>`, and a `stack<GameState>` as member variables
Recursion	`Game::spawnExplosion()` calls itself with a smaller particle count and reduced depth to create layered explosion effects

3 Implementation

3.1 Development Environment

- **Language:** C++17
- **Graphics & Audio library:** SFML 2.x (Simple and Fast Multimedia Library) — graphics, window, and audio modules
- **Build system:** CMake with `CMakePresets.json`
- **IDE:** Visual Studio Code
- **Version control:** Git / GitHub (`hamza-branch`)
- **Platform:** Windows 11

3.2 Encapsulation — The Player Class

OOP Concept: Encapsulation

Encapsulation means keeping a class's data private and only letting outside code interact with it through a controlled public interface. The class decides what can be read and what can be changed. Think of it like a vending machine: you press a button to get a result — you do not reach inside and grab the product directly.

The `Player` class is the clearest example of encapsulation in this project. Every field — position, HP, lives, timers — is `private`. The rest of the code, including `Game`, can only interact with the player through the public interface.

```
class Player {
private:
    // position, hp, maxHp, speed, lives, score
    // shootTimer, iframeTimer, shieldTimer, tilt, alive

public:
    // Getters: getPos(), getHp(), getLives(), getScore(), isAlive(),
        ...
    void takeDamage(float d);        // reduces hp
    void heal(float h);              // increases hp, capped at maxHp
    void loseLife();                 // decrements lives
    void clampToArena();             // keeps ship inside the play
        area
    Player& operator+=(int bonus);   // adds score (operator overload)
};
```

Listing 1: Player class skeleton (Player.h)

Why this matters: If `hp` were `public`, any part of the code could accidentally write `player.hp = -999`. Making it `private` turns that into a compile error. Only `takeDamage()` and `heal()` can modify health, and both do it safely.

3.3 Inheritance — Three Class Hierarchies

OOP Concept: Inheritance

Inheritance lets a child class reuse the code of a parent class and then add or override specific behaviour. The parent defines the shared interface and any common logic. Each child fills in the parts that are different. For example, a `Vehicle` base class could define that all vehicles have a speed; a `Car` child adds four wheels; a `Boat` child adds a hull.

3.3.1 Bullet Type Hierarchy

We need to tell four kinds of bullets apart at runtime without scattering raw integer comparisons through the code. Each bullet type is its own class:

```
class BulletType {          // abstract base -- defines the interface
    int id; bool playerBullet;
public:
    virtual ~BulletType() {}
    int getId() const; bool isPlayerBullet() const;
    static BulletType *PlayerNorm, *EnemyNorm, *EnemyBurst, *BossBeam
        ;
};
// Concrete subclasses -- each hard-codes its own id
class PlayerNormBullet : public BulletType { /* id=0, player bullet
    */ };
class EnemyNormBullet  : public BulletType { /* id=3, enemy bullet
    */ };
class EnemyBurstBullet : public BulletType { /* id=4, burst bullet
    */ };
class BossBeamBullet   : public BulletType { /* id=5, boss beam
    */ };
```

Listing 2: BulletType hierarchy skeleton (BulletType.h)

The four concrete objects are owned by a `BulletTypeRegistry` class defined at the bottom of `BulletType.h`. The static pointer definitions and registry creation are both in `main.cpp`. The registries are local variables at the top of `main()`, created *before* the `Game` object, so the singletons are valid for the entire programme lifetime:

```
class BulletTypeRegistry {
private:
    PlayerNormBullet playerNorm;   // owned by value -- no heap
        allocation
    EnemyNormBullet  enemyNorm;
    EnemyBurstBullet enemyBurst;
    BossBeamBullet   bossBeam;
public:
    BulletTypeRegistry() {
        BulletType::PlayerNorm  = &playerNorm;   // wire static
            pointers
        BulletType::EnemyNorm   = &enemyNorm;
        BulletType::EnemyBurst  = &enemyBurst;
        BulletType::BossBeam    = &bossBeam;
    }
    ~BulletTypeRegistry() {
        BulletType::PlayerNorm = nullptr;        // safe teardown at
            exit
        BulletType::EnemyNorm  = nullptr;
```

```
        BulletType::EnemyBurst = nullptr;
        BulletType::BossBeam   = nullptr;
    }
    BulletTypeRegistry(const BulletTypeRegistry&)            = delete;
    BulletTypeRegistry& operator=(const BulletTypeRegistry&) = delete;
};
```

Listing 3: BulletTypeRegistry — RAII singleton manager (BulletType.h)

```
// Static pointer definitions (must appear in exactly one .cpp file)
BulletType *BulletType::PlayerNorm = nullptr;
BulletType *BulletType::EnemyNorm  = nullptr;
BulletType *BulletType::EnemyBurst = nullptr;
BulletType *BulletType::BossBeam   = nullptr;

EnemyType *EnemyType::Small  = nullptr;
EnemyType *EnemyType::Medium = nullptr;
EnemyType *EnemyType::Boss   = nullptr;

const GameState GameState::Home(0);
// ... other GameState constants ...

int main() {
    BulletTypeRegistry registryBullet;  // constructor wires BulletType statics
    EnemyTypeRegistry  registry;         // constructor wires EnemyType statics
    try {
        Game game;
        game.init();
        game.run();
    } catch (...) { ... }
    // registries destroyed here -- destructors clear statics to nullptr
}
```

Listing 4: All static definitions and registry activation consolidated in main.cpp

Why not just use integers? Using class pointers means the compiler checks types. Writing `b.type == BulletType::PlayerNorm` is readable and safe. Writing `b.type == 0` would compile fine even if you accidentally typed `b.type == 100` — there is nothing to catch the mistake.

Why consolidate into main.cpp? C++ requires each static member variable to be *defined* in exactly one translation unit. Putting them all in `main.cpp` makes ownership clear and removes three separate `.cpp` files (`BulletType.cpp`, `EnemyType.cpp`, `GameState.cpp`), which simplifies the build. The registry objects are local variables in `main()`, so they are constructed before the game starts and destroyed when it ends — full lifetime control, no hidden global state.

3.3.2 Enemy Type Hierarchy

Enemy types follow the same pattern but carry meaningful gameplay data:

```
class EnemyType {       // abstract base -- stores gameplay stats
    // maxHp, shootInterval, scoreValue, moveSpeed, radius, kind
public:
    virtual ~EnemyType() {}
```

```
    // Getters: getMaxHp(), getShootInterval(), getScoreValue(), ...
    static EnemyType *Small, *Medium, *Boss;
};
// Stats baked in: (hp, shootInterval, score, speed, radius, kind)
class SmallEnemyType  : public EnemyType { /* 30 HP,   2.2s, 50 pts */ };
class MediumEnemyType : public EnemyType { /* 80 HP,   1.8s, 120 pts */ };
class BossEnemyType   : public EnemyType { /* 1200 HP, 0.8s, 2000 pts */ };
```

Listing 5: EnemyType hierarchy skeleton (EnemyType.h)

Singleton management: `EnemyType.h` also defines an `EnemyTypeRegistry` class that owns all three concrete instances and wires the static pointers in its constructor. It follows the same RAII pattern as `BulletTypeRegistry`. The registry is created as a local variable in `main()`, so it is alive for the full duration of the programme and automatically cleans up on exit.

3.3.3 Level Hierarchy

The `Level` base class defines the interface that every level must implement. Each concrete subclass fills in the specific details:

```
class Level {           // abstract base
protected:
    // name, desc, events (wave schedule), planetTexture

public:
    virtual ~Level() {}
    // Pure virtual -- every subclass MUST implement:
    virtual float       getDuration()          const = 0;
    virtual std::string getPlanetTexturePath() const = 0;
    virtual void        buildWaves()                  = 0;
    virtual void renderPlanet(sf::RenderWindow& w);  // has default impl.
};
class LevelOne   : public Level { /* 55s, planet1.png */ };
class LevelTwo   : public Level { /* 65s, planet2.png */ };
class LevelThree : public Level { /* 80s, planet3.png, two bosses */ };
```

Listing 6: Level abstract base class skeleton (Level.h)

3.4 Polymorphism — Same Call, Different Behaviour

OOP Concept: Polymorphism

Polymorphism means "many forms." If you have a base-class pointer and call a `virtual` function through it, C++ automatically calls the correct overridden version in the derived class — even though the pointer only knows about the base type. This decision happens at runtime, not at compile time, and is called *dynamic dispatch*.

In `Game.cpp`, levels are stored as base-class pointers. The game calls virtual functions through those pointers without ever checking which concrete level is loaded:

```
std::vector<Level *> levels;   // stores LevelOne*, LevelTwo*, or LevelThree*
```

```
2
3  // During initialisation -- all three calls go through the base
       pointer:
4  for (int i = 0; i < levels.size(); i++) {
5      levels[i]->buildWaves();    // calls LevelOne::buildWaves() OR
6                                  // LevelTwo::buildWaves() OR
                                       LevelThree::buildWaves()
7      levels[i]->loadPlanet();    // calls Level::loadPlanet() (base
           version)
8  }
9
10 // During gameplay rendering:
11 void Game::renderBackground() {
12     levels[currentLevel]->renderPlanet(window);  // correct planet
           drawn automatically
13 }
```

Listing 7: Polymorphic level calls in Game.cpp

The key insight: `Game` never writes `if (level == 1) do X; else if (level == 2) do Y`. The virtual dispatch table handles the routing automatically. Adding a fourth level later means writing one new subclass — no changes to `Game.cpp` at all.

3.5 Templates — The ObjectPool

OOP Concept: Templates

A template is a class or function where the type is a parameter. You write the logic once, and the compiler generates a correctly-typed version for each type you actually use it with. The code is the same; only the type changes. This avoids writing the same container or algorithm multiple times for different types.

The game can have up to 700 bullets, 80 enemies, and 1000 particles active at the same time. Calling `new` and `delete` every frame would be slow and could fragment memory. Instead we use a fixed-size *object pool*: a pre-allocated array where inactive slots are reused instead of freed.

```
1  template <class T, int N>
2  class ObjectPool
3  {
4  private:
5      std::array<T, N> items;    // fixed array -- allocated once, never
           reallocated
6
7  public:
8      ObjectPool() {
9          for (int i = 0; i < N; i++)
10             items[i].on = false;    // all slots start as "free"
11     }
12
13     int  capacity()  const { return N; }
14     T&   at(int i)         { return items[i]; }
15     const T& at(int i) const { return items[i]; }
16
17     // Find the first free slot and return a pointer to it
18     T* alloc() {
19         for (int i = 0; i < N; i++) {
20             if (!items[i].on)
```

```
                return &items[i];
            }
            return nullptr;    // pool is full -- caller must handle this
        }

        void clearAll() {
            for (int i = 0; i < N; i++)
                items[i].on = false;
        }

        int countActive() const {
            int c = 0;
            for (int i = 0; i < N; i++) if (items[i].on) c++;
            return c;
        }
};
```

Listing 8: ObjectPool template class (GameData.h)

The three instantiations in the `Game` class:

```
ObjectPool<Bullet,   MAX_BULLETS>   bullets;    // 700 slots
ObjectPool<Enemy,    MAX_ENEMIES>   enemies;    //  80 slots
ObjectPool<Particle, MAX_PARTICLES> particles;  // 1000 slots
```

Listing 9: Three pool instances in Game.h

The template is written once; the compiler generates three separate typed arrays from it. When a bullet is needed, `bullets.alloc()` finds a free slot and returns a pointer. When the bullet goes off-screen, `b.on = false` marks the slot as available again. No `new` or `delete` is called during gameplay.

3.6 Operator Overloading

OOP Concept: Operator Overloading

C++ lets you define what standard operators (+, =, ==, etc.) mean for your own classes. Instead of calling a method like `player.addScore(50)`, you can write `player += 50` and have it mean the same thing. It makes code easier to read when used carefully.

The project uses two operator overloads:

3.6.1 Player::operator+= (score addition)

```
// Instead of writing: player.addScore(e.scoreValue);
// We can write:       player += e.scoreValue;
Player& operator+=(int scoreBonus) {
    score += scoreBonus;
    return *this;
}
```

Listing 10: Score addition via operator overload (Player.h)

Used in `Game.cpp` when an enemy is destroyed:

```
if (e.hp <= 0.f) {
    e.on = false;
    player += e.scoreValue;   // clean, readable syntax
```

```
}
```

Listing 11: Using the overloaded operator in Game.cpp

3.6.2 GameState::operator== and operator!=

```
bool operator==(const GameState& o) const { return id == o.id; }
bool operator!=(const GameState& o) const { return id != o.id; }
```

Listing 12: Comparison operators on GameState (GameState.h)

Used throughout `Game.cpp` for readable state checks:

```
GameState current = stateStack.top();
if (current == GameState::Playing) { update(dt); }
```

Listing 13: State comparison in Game.cpp

3.7 Exception Handling — Custom Exception Classes

OOP Concept: Exception Handling

Exceptions let you signal that something went wrong without scattering error checks through every function. When an error occurs you `throw` an exception, and it propagates up the call stack until a `catch` block handles it. Defining custom exception classes (by inheriting from `std::exception`) lets you distinguish between different error types and handle each one appropriately.

We defined two custom exception classes inside `Game.cpp`:

```
// Thrown when a texture, font, or other asset cannot be loaded from disk
class AssetLoadException : public std::runtime_error {
public:
    AssetLoadException() : std::runtime_error("Failed to load asset") {}
};

// Thrown when startLevel() is called with an invalid index
class InvalidLevelException : public std::runtime_error {
public:
    InvalidLevelException() : std::runtime_error("Invalid level") {}
};
```

Listing 14: Custom exception classes (Game.cpp)

They are thrown and caught in `Game::init()` and `Game::startLevel()`:

```
// In Game::init() -- missing assets are non-fatal; the game warns and continues
try {
    loadTextureOrThrow(shipTexture, "assets/textures/spaceship.png");
    loadTextureOrThrow(smallEnemyTexture, "assets/textures/alien1.png");
    // ... more assets ...
} catch (const AssetLoadException& ex) {
    std::cerr << "Asset warning: " << ex.what() << "\n";
```

```
}

// In Game::startLevel() -- an invalid index is a programming bug
void Game::startLevel(int index) {
    if (index < 0 || index >= (int)levels.size())
        throw InvalidLevelException();
    // ...
}

// In main.cpp -- the outermost safety net
int main() {
    try {
        Game game;
        game.init();
        game.run();
    } catch (const std::exception& ex) {
        std::cerr << "Fatal error: " << ex.what() << std::endl;
        return 1;
    }
}
```

Listing 15: Throwing and catching exceptions (Game.cpp)

3.8 The Wave Spawn System

Enemies appear on screen through a three-stage pipeline. This section explains each stage in order.

3.8.1 Stage 1 — SpawnEvent: the schedule

When a level is loaded, `buildWaves()` fills a list of `SpawnEvent` objects. Each one is a row in the level's timetable:

```
// At t = 14.0 seconds, spawn 4 Small enemies starting at x=240,
// following the "sweep centre" path, 50px apart, 0.25s between each.
events.push_back(makeEvent(
    14.0f,              // fire at t=14 seconds into the level
    EnemyType::Small,   // what type of enemy
    sf::Vector2f(240, -40),  // group starting position (above screen)
    pathSweepCenter(),  // waypoints the group follows
    4,                  // 4 enemies in this group
    50.f,               // 50px horizontal spacing between them
    0.25f               // 0.25s delay between individual spawns
));
```

Listing 16: A single SpawnEvent (from LevelOne::buildWaves in Level.cpp)

At this point nothing has appeared on screen. This is only a schedule.

3.8.2 Stage 2 — checkSpawns(): firing events into the queue

Every frame, the game advances a timer and checks if any event's scheduled time has been reached:

```
void Game::checkSpawns(float dt) {
    std::vector<SpawnEvent>& evs = levels[currentLevel]->getEvents();
    for (int i = 0; i < evs.size(); i++) {
```

```
        SpawnEvent& ev = evs[i];
        if (!ev.fired && levelTimer >= ev.time) {
            ev.fired = true;
            for (int k = 0; k < ev.count; k++) {
                SpawnJob job;
                job.etype          = ev.etype;
                job.startPos.x    += (k - (ev.count-1)/2.f) * ev.
                    xSpacing;
                job.delayRemaining = k * ev.delay;  // stagger each
                    enemy
                job.path           = ev.path;
                spawnQueue.push(job);   // queued -- not yet alive
            }
        }
    }
}
```

Listing 17: Firing events from the schedule (Game.cpp)

The 4-enemy event above pushes 4 SpawnJobs with delays of 0s, 0.25s, 0.5s, and 0.75s, so the enemies appear one after the other.

3.8.3 Stage 3 — Draining the queue into live enemies

Each frame the front of the queue is checked. When its delay reaches zero, the enemy is activated in the object pool:

```
while (!spawnQueue.empty()) {
    SpawnJob& front = spawnQueue.front();
    front.delayRemaining -= dt;
    if (front.delayRemaining <= 0.f) {
        spawnEnemyFromJob(front);   // writes into an ObjectPool slot
        spawnQueue.pop();
    } else {
        break;  // rest of queue is still waiting
    }
}

void Game::spawnEnemyFromJob(const SpawnJob& job) {
    Enemy* e = enemies.alloc();   // grab a free pool slot
    if (!e) return;
    e->on           = true;       // mark the slot as active
    e->type         = job.etype;
    e->pos          = job.startPos;
    e->hp           = job.hp;
    e->path         = job.path;
    e->pathIndex    = 0;
    e->shootTimer   = job.shootInterval * randFloat(0.3f, 1.0f);
}
```

Listing 18: Activating a queued enemy (Game.cpp)

3.8.4 Enemy Movement along Waypoints

Once active, each enemy moves through a list of 2D waypoints:

```
if (e.pathIndex < e.path.size()) {
    sf::Vector2f target   = e.path[e.pathIndex];
```

```
    sf::Vector2f toTarget = target - e.pos;
    float dist = vlen(toTarget);      // vector length from Utils.h
    if (dist < 8.f)
        e.pathIndex++;                // close enough -- advance to
            next waypoint
    else {
        sf::Vector2f dir = vnorm(toTarget);   // normalise direction
        e.pos += dir * e.moveSpeed * dt;      // move toward waypoint
    }
} else {
    e.pos.y += 100.f * dt;            // ran out of waypoints -- drift
        off-screen
    if (e.pos.y > 800.f) e.on = false;
}
```

Listing 19: Waypoint movement (Game.cpp)

3.9 Recursion — Layered Explosion System

OOP Concept: Recursion

Recursion is when a function calls itself with a smaller version of the same problem until a base case is reached. It works well when a task naturally breaks down into identical subtasks. Here, spawning an explosion that itself spawns smaller explosions is a natural fit.

When an enemy is destroyed, `Game::spawnExplosion()` calls itself recursively to create layered particle bursts. Each recursive call spawns a smaller explosion at a nearby random offset, building up a chain of progressively smaller blasts:

```
void Game::spawnExplosion(sf::Vector2f pos, sf::Color cStart,
                          sf::Color cEnd, int count, int depth) {
    // Base step: spawn particles at this position
    for (int i = 0; i < count; i++) {
        Particle* p = particles.alloc();
        if (!p) return;
        // ... set pos, vel, color, life
        p->on = true;
    }
    // Recursive step: spawn a smaller explosion nearby
    if (depth > 0) {
        float ox = randFloat(-20.f, 20.f);
        float oy = randFloat(-20.f, 20.f);
        spawnExplosion(pos + sf::Vector2f(ox, oy),
                       cStart, cEnd,
                       count / 2,   // fewer particles each time
                       depth - 1);  // base case: depth == 0
    }
}
```

Listing 20: Recursive explosion spawner (Game.cpp)

Base case: `depth == 0` — the function stops. **Recursive case:** each call spawns particles, then calls itself with `count/2` and `depth-1`. A depth-2 explosion ends up placing particles at three positions, each burst smaller than the last.

3.10 Game Loop and State Machine

The `GameState` class works with a `std::stack` to implement a push/pop state machine. When the player pauses, the Paused state is pushed on top of Playing. When they resume, it is popped and Playing is active again. This means overlay states do not destroy the state underneath them.

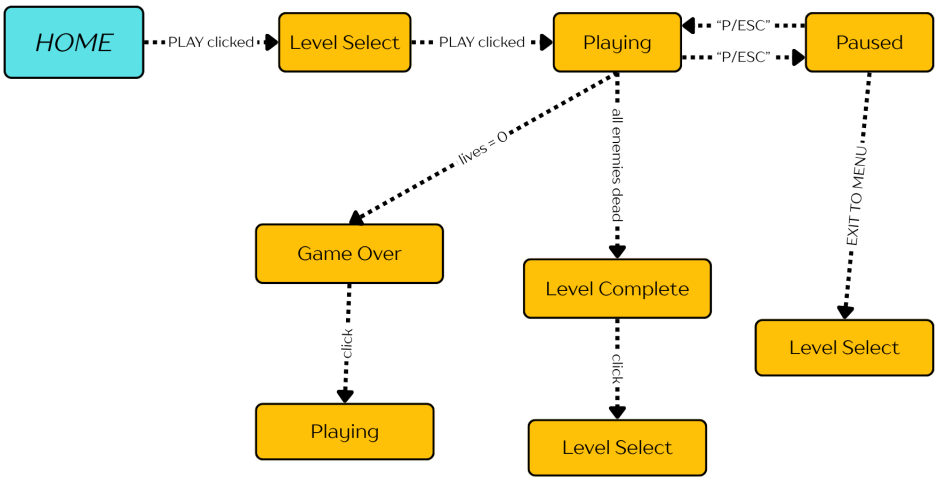

Figure 3: Game state transitions. Arrows show what action triggers each transition.

4 Testing

4.1 Test Cases

Table 5: Edge Case and Exception Test Cases

TC	Scenario	Expected Outcome	Pass?
TC-01	Hold Space to fire ~700 bullets	Pool fills up; `alloc()` returns `nullptr`; no crash or out-of-bounds	✓
TC-02	Take damage immediately after a previous hit (iframes active)	Damage ignored for iframe duration; HP unchanged	✓
TC-03	Rename or delete a texture file; launch game	`AssetLoadException` caught; warning printed; game continues	✓
TC-04	Call `startLevel(-1)` programmatically	`InvalidLevelException` thrown and caught at outermost level in `main()`	✓
TC-05	Kill all enemies including boss	`isLevelClear()` returns true; Level Complete overlay shown	✓

4.2 Memory Leak Testing

An earlier version used `new` to create the `BulletType` and `EnemyType` singletons. Since `delete` was never called on them, memory analysis tools reported leaks. We fixed this by introducing `BulletTypeRegistry` and `EnemyTypeRegistry` classes. Each registry stores its concrete instances as plain member variables — no heap allocation — wires the static pointers in its constructor, and resets them to `nullptr` in its destructor. Both registry objects are local variables at the top of `main()`, so they live for exactly as long as the programme runs and are cleaned up automatically when `main()` returns. We also moved all static member definitions into `main.cpp`, which removed the need for the separate `BulletType.cpp`, `EnemyType.cpp`, and `GameState.cpp` files entirely.

5 Expected Output — Screenshots

The screenshots below were taken from the running game. Each one shows a different screen or game state.

Figure 4: Home screen: title text, scrolling star background, two decorative planets, and the PLAY button.

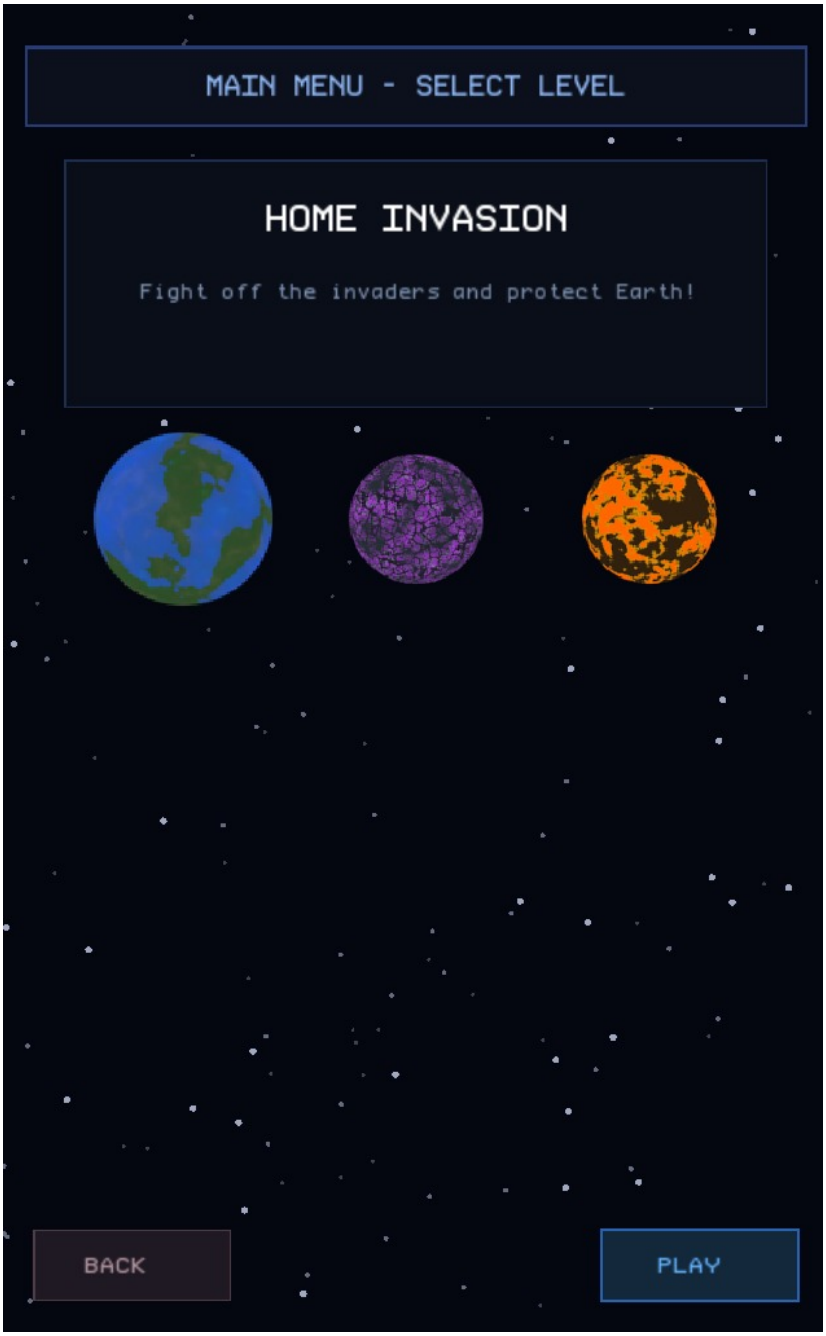

Figure 5: Level Select screen: three planet buttons (the selected one is larger), level name and description preview, BACK and PLAY buttons.

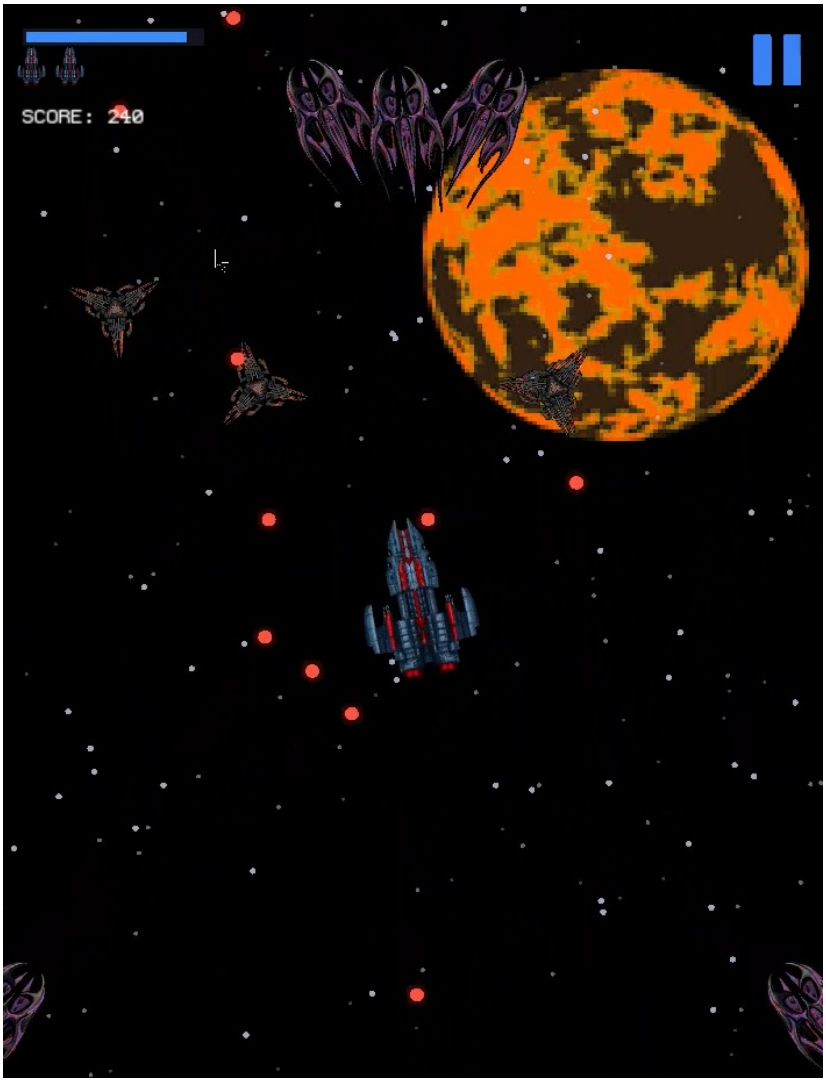

Figure 6: Mid-game: player ship, enemy formation following waypoints, player bullets, HUD showing HP bar, lives, and score.

Figure 7: Boss encounter: large boss sprite on screen, red boss HP bar centred at the top, particle explosions visible.

Figure 8: Game Over overlay: dimmed background, “GAME OVER” heading, final score, and “Click to restart” prompt.

Figure 9: Level Complete overlay: green panel with level name, score, and “Click to continue” prompt.

6 Results and Discussion

6.1 Results

- The game runs smoothly at 60 frames per second on standard hardware.
- All three levels are completable: Level One (55s), Level Two (65s), Level Three (80s, two bosses).
- Background music plays via `sf::Music` using OGG audio streaming.
- The `ObjectPool` successfully prevents runtime allocation: no `new` or `delete` occurs during gameplay.
- The three inheritance hierarchies (bullets, enemies, levels) are genuinely polymorphic — `Game.cpp` never queries concrete types to decide which base-class method to call.
- The state machine handles all screen transitions cleanly, including nested states (Paused inside Playing).
- Rendering logic is fully separated into `Game_graphics.cpp`, keeping `Game.cpp` focused on game logic only.

6.2 Analysis

- **ObjectPool vs dynamic allocation:** Checking 700 slots linearly to find a free bullet takes at most 700×1 comparisons per allocation. At 60 fps, with approximately 10 allocations per second, this is negligible. The benefit — zero fragmentation, cache-friendly memory layout — outweighs the linear scan.
- **Polymorphism overhead:** Virtual function calls add one pointer dereference per call (the vtable lookup). For the small number of calls per frame (3 level methods, up to 80 enemy type lookups), this overhead is unmeasurable.
- **Template code size:** The compiler generates three separate versions of `ObjectPool`. For three small instantiations, the binary size increase is negligible.

7 Challenges and Solutions

7.1 Pointer Leaks from Static Singletons

Problem: The first version used `new` to create the `BulletType` and `EnemyType` singleton objects. Since `delete` was never called on them, memory tools reported leaks.

Solution: We introduced `BulletTypeRegistry` and `EnemyTypeRegistry` using RAII. Each registry stores its concrete instances as plain member variables (no heap allocation). The constructor wires the static pointers; the destructor clears them to `nullptr`. Both are declared as local variables at the top of `main()`, which gives full lifetime control and removes the need for three separate `.cpp` files. Copy construction and assignment are `= deleted` to prevent accidental duplication.

Learning: Put resource setup and teardown inside a class using RAII. Declaring the registry as a local in `main()` is the cleanest approach — it lives for exactly as long as the programme runs, with no static initialisation order problems.

7.2 Enemies All Spawning at the Same Position

Problem: When a wave of 4 enemies was triggered, all 4 appeared at exactly the same (x, y) coordinate and stacked on top of each other.

Solution: We added an `xSpacing` field to `SpawnEvent`. In `checkSpawns`, each enemy's starting x is offset by `(k - (count-1)/2.0) * xSpacing`, which centres the group around the event's `startPos`.

Learning: Centre-offset arithmetic — shifting by half the total span — is a standard way to distribute a group of elements evenly around a point, whether in a game or a UI layout.

7.3 Frame-rate-dependent Movement

Problem: Early on, objects moved a fixed number of pixels per frame. On a faster machine running at 120 fps the game ran twice as fast as on a 60 fps machine.

Solution: All movement and timer updates are multiplied by `dt` (delta time in seconds). At 60 fps, `dt` $\approx$ `0.0167s`; at 120 fps, `dt` $\approx$ `0.0083s`. The product gives the same distance per second regardless of frame rate.

Learning: Always measure game speed in units per *second*, not per *frame*. Frame rate is not a reliable unit of time.

8 Future Work

- **Power-ups:** Add a `PowerUp` base class with subclasses for shield, speed boost, and rapid fire. This would follow the same inheritance pattern already used by `BulletType`.
- **High score persistence:** Save the player's score to a `.txt` file on game over and show the top 5 scores on the home screen.
- **Sound effects:** Use SFML's `sf::SoundBuffer` and `sf::Sound` to add shoot, explosion, and level-complete sounds. Background music is already working; effects are the next step.
- **More levels:** The level system is built for extension. Adding `LevelFour` only requires a new subclass of `Level` — no changes to `Game.cpp`.
- **Unit testing:** Use Google Test to test `ObjectPool` allocation and deallocation, `Player` damage and healing invariants, and spawn-queue ordering.

9 Conclusion

Deepspace Defenders shows that a complete, interactive application — with real-time graphics, multiple screens, and hundreds of live game objects — can be built cleanly using the core OOP concepts taught in this course.

We applied every required OOP concept in a way that actually made the code better, not just to tick a box:

- **Encapsulation** kept the player's internal state safe. Private fields mean no other code can accidentally corrupt health or lives.
- **Inheritance** let three separate type hierarchies (bullets, enemies, levels) share common structure while each subclass handles its own specific behaviour.
- **Polymorphism** meant the game loop never needs to know which level or enemy type is active. Virtual dispatch handles it, and adding a new type requires no changes to existing code.
- **Templates** gave us a single `ObjectPool` that works for bullets, enemies, and particles — written once, compiled three times.
- **Operator overloading** made score addition and state comparison read naturally in the code.
- **Custom exceptions** kept error-handling logic separate from normal game logic.

The main thing we learned is that OOP concepts are not useful because they are on the syllabus — they are useful because each one solves a real problem. Every concept above was added because it made the code shorter, safer, or easier to change. That is what OOP is actually for.

References

[1] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.

[2] S. Meyers, *Effective Modern C++*. O'Reilly Media, 2014.

[3] *SFML Documentation*. [Online]. Available: `https://www.sfml-dev.org/documentation/`. [Accessed: May 10, 2026].

[4] *cppreference.com — C++ Reference*. [Online]. Available: `https://en.cppreference.com`. [Accessed: May 10, 2026].

[5] *Game Programming Patterns* by Robert Nystrom. [Online]. Available: `https://gameprogrammingpatter` `com`. [Accessed: May 10, 2026].

A Complete Source Code

A.1 GameData.h

```
#pragma once
#include <SFML/Graphics.hpp>
#include <array>
#include <vector>
#include "BulletType.h"
#include "EnemyType.h"

struct Bullet  { sf::Vector2f pos, vel; float dmg; BulletType* type; bool on; };
struct Enemy   { sf::Vector2f pos, vel; float hp, maxHp; EnemyType* type;
                 float shootTimer, shootInterval, angle, pulse, moveSpeed;
                 int scoreValue, pathIndex; std::vector<sf::Vector2f> path; bool on; };
struct Particle{ sf::Vector2f pos, vel; sf::Color colorStart, colorEnd;
                 float life, maxLife, size; bool on; };
struct SpawnEvent { float time, xSpacing, delay; EnemyType* etype;
                    sf::Vector2f startPos; std::vector<sf::Vector2f> path;
                    int count; bool fired; };
struct SpawnJob   { EnemyType* etype; sf::Vector2f startPos;
                    float delayRemaining, hp, maxHp, shootInterval, moveSpeed;
                    int scoreValue; std::vector<sf::Vector2f> path; };
struct Star    { sf::Vector2f pos; float speed, brightness, size; };

const int MAX_BULLETS=700, MAX_ENEMIES=80, MAX_PARTICLES=1000, MAX_STARS=200;

template <class T, int N>
class ObjectPool {
    std::array<T, N> items;
public:
    ObjectPool() { for (int i=0;i<N;i++) items[i].on=false; }
    int  capacity()  const { return N; }
    T&   at(int i)         { return items[i]; }
    T*   alloc()  { for(int i=0;i<N;i++) if(!items[i].on) return &items[i]; return nullptr; }
    void clearAll()  { for(int i=0;i<N;i++) items[i].on=false; }
    int  countActive() const { int c=0; for(int i=0;i<N;i++) if(items[i].on) c++; return c; }
};
```

Listing 21: GameData.h — all data structs and ObjectPool template

A.2 Game.h

```
#pragma once
#include <SFML/Graphics.hpp>
#include <SFML/Audio.hpp>
#include <stack>
```

```
#include <vector>
#include <queue>
#include "GameState.h"
#include "Player.h"
#include "GameData.h"
#include "Level.h"
#include "Utils.h"

class Game {
private:
    sf::RenderWindow window;
    sf::Font font;
    sf::Music music;
    sf::Texture shipTexture, smallEnemyTexture, mediumEnemyTexture,
                bossEnemyTexture, homeplanet1, homeplanet2,
                   homeplanet3, pauseIcon;
    sf::Sprite shipSprite, pauseSprite;
    sf::View gameView;

    std::stack<GameState> stateStack;
    Player player;
    ObjectPool<Bullet,   MAX_BULLETS>   bullets;
    ObjectPool<Enemy,    MAX_ENEMIES>   enemies;
    ObjectPool<Particle, MAX_PARTICLES> particles;
    std::array<Star, MAX_STARS> stars;
    std::vector<Level*> levels;
    std::queue<SpawnJob> spawnQueue;
    int currentLevel, selectedLevel;
    float levelTimer, bgY;
    bool bossActive;

    // Core game methods
    void handleEvents(); void update(float dt); void render();
    void startLevel(int index); void resetPlayer();
    void checkCollisions(); void checkSpawns();
    void spawnPlayerBullet(); void spawnEnemyBullet(Enemy& e);
    void spawnEnemyFromJob(const SpawnJob& job);
    void spawnExplosion(sf::Vector2f pos, sf::Color cs, sf::Color ce,
                        int count, int depth=1);
    // Render methods (implemented in Game_graphics.cpp)
    void renderBackground(); void renderStars(); void renderPlayer();
    void renderEnemies(); void renderBullets(); void renderParticles
       ();
    void renderHUD(); void renderBossHP();
    void renderHomeScreen(); void renderLevelSelect();
    void renderPauseOverlay(); void renderGameOverOverlay();
    void renderLevelCompleteOverlay();
    void drawPlayerShip(sf::Vector2f pos, float tilt, float scale);
    void drawSmallEnemy(sf::Vector2f pos, float angle);
    void drawMediumEnemy(sf::Vector2f pos, float angle);
    void drawBossEnemy(sf::Vector2f pos, float angle);
    void initStars(); void buildLevels();
    void drawTextCentered(const std::string& str, float y, int size,
       sf::Color col);
    bool isLevelClear(); int countActiveEnemies();

public:
    Game(); ~Game();
```

```
60      void init(); void run();
61  };
```

Listing 22: Game.h — Game class declaration

A.3 Level.h

```
1   #pragma once
2   #include <SFML/Graphics.hpp>
3   #include <string>
4   #include <vector>
5   #include "GameData.h"
6
7   class Level {
8   protected:
9       std::string name, desc;
10      std::vector<SpawnEvent> events;
11      sf::Texture planetTexture;
12      bool planetLoaded;
13  public:
14      Level() : planetLoaded(false) {}
15      virtual ~Level() {}
16      std::string getName() const { return name; }
17      std::string getDesc() const { return desc; }
18      std::vector<SpawnEvent>& getEvents() { return events; }
19      virtual float       getDuration()          const = 0;
20      virtual std::string getPlanetTexturePath() const = 0;
21      virtual void        buildWaves()                 = 0;
22      virtual void renderPlanet(sf::RenderWindow& window);
23      void loadPlanet();
24  };
25
26  class LevelOne   : public Level { public: LevelOne();
27      float getDuration() const override { return 55.f; }
28      std::string getPlanetTexturePath() const override;
29      void buildWaves() override; };
30
31  class LevelTwo   : public Level { public: LevelTwo();
32      float getDuration() const override { return 65.f; }
33      std::string getPlanetTexturePath() const override;
34      void buildWaves() override; };
35
36  class LevelThree : public Level { public: LevelThree();
37      float getDuration() const override { return 80.f; }
38      std::string getPlanetTexturePath() const override;
39      void buildWaves() override; };
```

Listing 23: Level.h — abstract Level base and concrete subclasses

A.4 Utils.h

```
1   #pragma once
2   #include <SFML/Graphics.hpp>
3   #include <cmath>
4   #include <cstdlib>
5
6   inline float vlen(sf::Vector2f v) {
```

```
    return std::sqrt(v.x * v.x + v.y * v.y);
}
inline sf::Vector2f vnorm(sf::Vector2f v) {
    float l = vlen(v);
    if (l < 0.0001f) return sf::Vector2f(0.f, 0.f);
    return sf::Vector2f(v.x / l, v.y / l);
}
inline float lerp(float a, float b, float t) { return a + (b - a) * t
    ; }
inline sf::Color colorLerp(sf::Color a, sf::Color b, float t) {
    if (t < 0.f) t = 0.f;
    if (t > 1.f) t = 1.f;
    return sf::Color(
        (sf::Uint8)(a.r + (b.r - a.r) * t),
        (sf::Uint8)(a.g + (b.g - a.g) * t),
        (sf::Uint8)(a.b + (b.b - a.b) * t),
        (sf::Uint8)(a.a + (b.a - a.a) * t));
}
inline float clampf(float val, float lo, float hi) {
    if (val < lo) return lo;
    if (val > hi) return hi;
    return val;
}
inline float randFloat(float lo, float hi) {
    float r = (float)rand() / (float)RAND_MAX;
    return lo + r * (hi - lo);
}
inline int randInt(int lo, int hi) {
    if (hi <= lo) return lo;
    return lo + rand() % (hi - lo + 1);
}
```

Listing 24: Utils.h — math utility functions

A.5 BulletType.h

```
#pragma once

class BulletType {
    int  id;
    bool playerBullet;
public:
    BulletType(int id, bool playerBullet) : id(id), playerBullet(
        playerBullet) {}
    virtual ~BulletType() {}
    int  getId()          const { return id; }
    bool isPlayerBullet() const { return playerBullet; }
    static BulletType *PlayerNorm;
    static BulletType *EnemyNorm;
    static BulletType *EnemyBurst;
    static BulletType *BossBeam;
};
class PlayerNormBullet : public BulletType {
public: PlayerNormBullet() : BulletType(0, true)  {} };
class EnemyNormBullet  : public BulletType {
public: EnemyNormBullet()  : BulletType(3, false) {} };
class EnemyBurstBullet : public BulletType {
public: EnemyBurstBullet() : BulletType(4, false) {} };
```

```
22 class BossBeamBullet   : public BulletType {
23 public: BossBeamBullet()   : BulletType(5, false) {} };
24
25 class BulletTypeRegistry {
26 private:
27     PlayerNormBullet playerNorm;
28     EnemyNormBullet  enemyNorm;
29     EnemyBurstBullet enemyBurst;
30     BossBeamBullet   bossBeam;
31 public:
32     BulletTypeRegistry() {
33         BulletType::PlayerNorm  = &playerNorm;
34         BulletType::EnemyNorm   = &enemyNorm;
35         BulletType::EnemyBurst  = &enemyBurst;
36         BulletType::BossBeam    = &bossBeam;
37     }
38     ~BulletTypeRegistry() {
39         BulletType::PlayerNorm = nullptr;
40         BulletType::EnemyNorm  = nullptr;
41         BulletType::EnemyBurst = nullptr;
42         BulletType::BossBeam   = nullptr;
43     }
44     // preventing copying
45     BulletTypeRegistry(const BulletTypeRegistry&)            = delete
           ;
46     BulletTypeRegistry& operator=(const BulletTypeRegistry&) = delete
           ;
47 };
```

Listing 25: BulletType.h — bullet type hierarchy and registry

A.6 EnemyType.h

```
1  #pragma once
2
3  class EnemyType {
4      float maxHp, shootInterval, moveSpeed, radius;
5      int   scoreValue, kind;
6  public:
7      EnemyType(float maxHp, float shootInterval, int scoreValue,
8                float moveSpeed, float radius, int kind)
9          : maxHp(maxHp), shootInterval(shootInterval), scoreValue(
               scoreValue),
10           moveSpeed(moveSpeed), radius(radius), kind(kind) {}
11     virtual ~EnemyType() {}
12     float getMaxHp()         const { return maxHp; }
13     float getShootInterval() const { return shootInterval; }
14     int   getScoreValue()    const { return scoreValue; }
15     float getMoveSpeed()     const { return moveSpeed; }
16     float getRadius()        const { return radius; }
17     int   getKind()          const { return kind; }
18     static EnemyType *Small;
19     static EnemyType *Medium;
20     static EnemyType *Boss;
21 };
22 class SmallEnemyType  : public EnemyType {
23 public: SmallEnemyType()  : EnemyType( 30.f, 2.2f,   50, 200.f, 16.f,
       0) {} };
```

```
class MediumEnemyType : public EnemyType {
public: MediumEnemyType() : EnemyType( 80.f, 1.8f,  120, 160.f, 22.f, 1) {} };
class BossEnemyType   : public EnemyType {
public: BossEnemyType()   : EnemyType(1200.f, 0.8f, 2000,  80.f, 60.f, 2) {} };

class EnemyTypeRegistry {
private:
    SmallEnemyType  small;
    MediumEnemyType medium;
    BossEnemyType   boss;
public:
    EnemyTypeRegistry() {
        EnemyType::Small  = &small;
        EnemyType::Medium = &medium;
        EnemyType::Boss   = &boss;
    }
    ~EnemyTypeRegistry() {
        EnemyType::Small  = nullptr;
        EnemyType::Medium = nullptr;
        EnemyType::Boss   = nullptr;
    }
    // Prevent copying
    EnemyTypeRegistry(const EnemyTypeRegistry&)            = delete;
    EnemyTypeRegistry& operator=(const EnemyTypeRegistry&) = delete;
};
```

Listing 26: EnemyType.h — enemy type hierarchy and registry

A.7 GameState.h

```
#pragma once
class GameState {
    int id;
public:
    GameState(int i = 0) : id(i) {}
    int  getId() const { return id; }
    bool operator==(const GameState& o) const { return id == o.id; }
    bool operator!=(const GameState& o) const { return id != o.id; }
    static const GameState Home;
    static const GameState LevelSelect;
    static const GameState Playing;
    static const GameState Paused;
    static const GameState GameOver;
    static const GameState LevelComplete;
};
```

Listing 27: GameState.h — game screen state (constants defined in main.cpp)

A.8 Player.h

```
#pragma once
#include <SFML/Graphics.hpp>
#include "Utils.h"

class Player {
```

```
6  private:
7      sf::Vector2f pos;
8      float hp, maxHp, speed;
9      int   lives, score;
10     float shootTimer, shootInterval, iframeTimer, shieldTimer, tilt;
11     bool  alive;
12 public:
13     Player() : pos(240.f,600.f), hp(100.f), maxHp(100.f), speed(310.f
           ),
14                lives(3), score(0), shootTimer(0.f), shootInterval
                   (0.10f),
15                iframeTimer(0.f), shieldTimer(0.f), tilt(0.f), alive(
                   true) {}
16
17     // Getters
18     sf::Vector2f getPos()           const { return pos; }
19     float        getHp()            const { return hp; }
20     float        getMaxHp()         const { return maxHp; }
21     float        getSpeed()         const { return speed; }
22     int          getLives()         const { return lives; }
23     int          getScore()         const { return score; }
24     float        getShootTimer()    const { return shootTimer; }
25     float        getShootInterval()const { return shootInterval; }
26     float        getIframeTimer()   const { return iframeTimer; }
27     float        getShieldTimer()   const { return shieldTimer; }
28     float        getTilt()          const { return tilt; }
29     bool         isAlive()          const { return alive; }
30
31     // Setters
32     void setPos(sf::Vector2f p)  { pos = p; }
33     void setTilt(float t)        { tilt = t; }
34     void setShootTimer(float t)  { shootTimer = t; }
35     void setIframeTimer(float t) { iframeTimer = t; }
36     void setShieldTimer(float t) { shieldTimer = t; }
37     void setAlive(bool a)        { alive = a; }
38
39     // Behaviour
40     void move(sf::Vector2f delta) { pos += delta; }
41     void clampToArena() {
42         pos.x = clampf(pos.x, 24.f, 456.f);
43         pos.y = clampf(pos.y, 24.f, 696.f);
44     }
45     void takeDamage(float d) { hp -= d; }
46     void heal(float h) { hp = (hp + h > maxHp) ? maxHp : (hp + h); }
47     void addScore(int s) { score += s; }
48     void loseLife()      { lives--; }
49     void tickShootTimer(float dt) { shootTimer  -= dt; }
50     void tickIframe(float dt)     { iframeTimer -= dt; }
51     void tickShield(float dt)     { shieldTimer -= dt; }
52
53     void resetForLevel() {
54         pos = sf::Vector2f(240.f, 600.f);
55         hp = maxHp;  speed = 310.f;
56         shootTimer = iframeTimer = shieldTimer = tilt = 0.f;
57         alive = true;  score = 0;  lives = 3;
58     }
59
60     // Operator overload: player += scoreValue
```

```
61      Player& operator+=(int scoreBonus) { score += scoreBonus; return
            *this; }
62  };
```

Listing 28: Player.h — fully encapsulated player class

A.9 main.cpp

```
1   #include "Game.h"
2   #include <iostream>
3   #include "BulletType.h"
4   #include "EnemyType.h"
5   #include "GameState.h"
6
7   // GameState static constant definitions (previously in GameState.cpp
        )
8   const GameState GameState::Home(0);
9   const GameState GameState::LevelSelect(1);
10  const GameState GameState::Playing(2);
11  const GameState GameState::Paused(3);
12  const GameState GameState::GameOver(4);
13  const GameState GameState::LevelComplete(5);
14
15  // BulletType static pointer definitions (previously in BulletType.
        cpp)
16  BulletType *BulletType::PlayerNorm = nullptr;
17  BulletType *BulletType::EnemyNorm  = nullptr;
18  BulletType *BulletType::EnemyBurst = nullptr;
19  BulletType *BulletType::BossBeam   = nullptr;
20
21  // EnemyType static pointer definitions (previously in EnemyType.cpp)
22  EnemyType *EnemyType::Small  = nullptr;
23  EnemyType *EnemyType::Medium = nullptr;
24  EnemyType *EnemyType::Boss   = nullptr;
25
26  int main() {
27      // Registries created as locals -- alive for the entire main(),
            then auto-destroyed
28      BulletTypeRegistry registryBullet;
29      EnemyTypeRegistry  registry;
30
31      try {
32          Game game;
33          game.init();
34          game.run();
35      } catch (const std::exception& ex) {
36          std::cerr << "Fatal error: " << ex.what() << std::endl;
37          return 1;
38      }
39      return 0;
40  }
```

Listing 29: main.cpp — static definitions, registry activation, and entry point

A.10 Game_graphics.cpp (key excerpt)

```
1   // render() dispatches to the correct sub-renderer based on current
        state
```

```cpp
void Game::render() {
    window.setView(window.getDefaultView());
    window.clear(sf::Color::Black);
    window.setView(gameView);

    GameState current = stateStack.top();

    if (current == GameState::Home)      { renderHomeScreen(); return; }
    if (current == GameState::LevelSelect){ renderLevelSelect(); return; }

    renderBackground();
    renderStars();
    renderEnemies();
    renderBullets();
    renderParticles();
    renderPlayer();
    renderHUD();
    if (bossActive) renderBossHP();

    if (current == GameState::Paused)        renderPauseOverlay();
    if (current == GameState::GameOver)      renderGameOverOverlay();
    if (current == GameState::LevelComplete)
        renderLevelCompleteOverlay();
}

// renderBullets() uses BulletType pointers for type-safe branching
void Game::renderBullets() {
    sf::RenderStates glowState;
    glowState.blendMode = sf::BlendAdd;
    for (int i = 0; i < bullets.capacity(); i++) {
        Bullet& b = bullets.at(i);
        if (!b.on || !b.type) continue;
        if (b.type->isPlayerBullet()) {
            // Draw player bullet as a cyan streak + soft glow
            sf::RectangleShape streak(sf::Vector2f(2.f, 12.f));
            streak.setPosition(b.pos);
            streak.setFillColor(sf::Color(150, 230, 255));
            window.draw(streak);
        } else if (b.type == BulletType::BossBeam) {
            // Draw boss beam rotated in the direction of travel
            sf::RectangleShape beam(sf::Vector2f(5.f, 14.f));
            float angle = std::atan2(b.vel.y, b.vel.x) * 180.f /
                3.14159f - 90.f;
            beam.setRotation(angle);
            beam.setFillColor(sf::Color(140, 180, 255));
            window.draw(beam);
        }
        // else: enemy normal/burst bullets drawn as red orbs
    }
}
```

Listing 30: Game_graphics.cpp — render() dispatcher and renderBullets() (excerpt)

A.11 Game.cpp (key excerpts)

```cpp
// Custom exception classes
```

```
class AssetLoadException : public std::runtime_error {
public: AssetLoadException() : std::runtime_error("Failed to load
    asset") {} };

class InvalidLevelException : public std::runtime_error {
public: InvalidLevelException() : std::runtime_error("Invalid level")
     {} };

// Asset loading with exception handling
void Game::init() {
    // ... window creation ...
    try {
        loadTextureOrThrow(shipTexture,      "assets/textures/
            spaceship.png");
        loadTextureOrThrow(smallEnemyTexture,"assets/textures/alien1.
            png");
        // ... more assets ...
    } catch (const AssetLoadException& ex) {
        std::cerr << "Asset warning: " << ex.what() << "\n";
    }
    if (!music.openFromFile("assets/audio/music1.ogg"))
        std::cerr << "Music load failed\n";
    else { music.setLoop(true); music.play(); }
    buildLevels();
    initStars();
    stateStack.push(GameState::Home);
}

// Level guard with exception
void Game::startLevel(int index) {
    if (index < 0 || index >= (int)levels.size())
        throw InvalidLevelException();
    currentLevel = index;
    // reset player, clear pools, reset timer ...
}

// Collision detection sketch
void Game::checkCollisions() {
    for (int i = 0; i < bullets.capacity(); i++) {
        Bullet& b = bullets.at(i);
        if (!b.on) continue;
        if (b.type->isPlayerBullet()) {
            for (int j = 0; j < enemies.capacity(); j++) {
                Enemy& e = enemies.at(j);
                if (!e.on) continue;
                float dist = vlen(b.pos - e.pos);
                if (dist < e.type->getRadius()) {
                    e.hp -= b.dmg; b.on = false;
                    if (e.hp <= 0.f) {
                        e.on = false;
                        player += e.scoreValue;   // operator+=
                            overload
                        spawnExplosion(e.pos, ...);
                    }
                }
            }
        }
        // else: check player collision with enemy bullets
```

```
55         }
56 }
```

Listing 31: Game.cpp — custom exceptions, init, and collision skeleton

B Build and Run Instructions

B.1 Prerequisites

- CMake 3.15 or later
- A C++17 compiler (MSVC 2019+, GCC 9+, or Clang 10+)
- SFML 2.x (fetched automatically by CMake via `FetchContent`)

B.2 Building with CMake

```
# From the project root directory
cmake --preset debug          # configure (creates build/debug/)
cmake --build build/debug     # compile

# The executable appears at build/debug/SpaceShooter
```

Listing 32: Configure and build

B.3 Running the Game

```
# Run from the project root so asset paths resolve correctly
./build/debug/SpaceShooter
```

Listing 33: Running

B.4 Controls

Key / Input	Action
W / A / S / D or Arrow keys	Move ship
Space	Fire bullet
P or Escape	Pause / Resume
Left mouse click	Navigate menus

Table 6: Player controls